HACXLERATE 2026 HACKATHON · TECHNICAL SPECIFICATION DOSSIER

SCAMTRACE

AI-Powered Scam Detection, Brand Impersonation Defense & Threat Telemetry Platform

Dual-Portal Architecture: Public Protection Portal (SCAMTRACE Protect) & SecOps Intelligence Desk
Core Forensic Pillars: Heuristic Rule Engine · Perceptual dHash Comparison · Authoritative Registry Lookup
Version: 1.0.0 Production Architecture · **Date:** October 2026 · **Track:** Cybersecurity & AI

EVIDENCE-LED DETECTION	AUTHORITATIVE LOGO CHECK	ENTERPRISE SECOPS PORTAL
Every risk signal grounds directly in extracted evidence without unverifiable AI hallucination.	Automated Wikidata & Commons discovery; 3-factor dHash, palette, & OCR text difference scoring.	Dedicated administrator & analyst workspace with triage queue, case notes, and audit logs.

CONFIDENTIALITY NOTICE: Prepared exclusively for HacXLerate 2026 hackathon evaluation. Contains architectural schematics, forensic algorithms, database models, and verified execution logs. All documented features and endpoints reflect the active repository codebase.

Table of Contents & Executive Summary

This specification report covers 16 core sections documenting the complete SCAMTRACE security platform:

Sec	Title	Description	Sec	Title	Description
1	**Executive Summary**	Philosophy & Evidence Loop	9	**Detection Methods**	dHash, Palette & Rulebook
2	**Problem & Objectives**	Phishing & Impersonation	10	**AI & External Data**	NVIDIA, OCR, Wikidata
3	**Implemented Features**	Full Module Inventory	11	**Setup & Execution**	Install, Seed & Run
4	**Technology Stack**	Package Manifest & Versions	12	**Verified Testing**	E2E Execution Results
5	**System Architecture**	Topology & Data Flow	13	**Security & Privacy**	Redaction, Cold Starts
6	**User Workflows**	Forensic Lifecycle Diagram	14	**Deployment Topology**	Vercel & Render Pipelines
7	**Backend APIs**	REST Endpoint Specs	15	**Limitations & Roadmap**	Known Limits & Next Steps
8	**Database Schema**	PostgreSQL Models & ERD	16	**Conclusion**	Hackathon Verdict Summary

1. Executive Summary & Core Philosophy

SCAMTRACE is an explainable, multi-modal scam intelligence and digital brand protection system developed for the **HacXLerate 2026** hackathon. The application tackles the exploding crisis of digital impersonation, phishing, counterfeit mobile applications, and manipulated brand identities. Unlike conventional security tools that act as opaque black boxes, SCAMTRACE is built on an **evidence-led transparency philosophy**: every risk rating is accompanied by exact ground-truth evidence, extracted psychological persuasion tactics, and concrete next steps.

The differentiator is the complete evidence loop: a user submits a report, the system extracts indicators, compares those clues with known reports, explains matches and uncertainty, routes the report for review, and updates public intelligence when its status changes. Both portals are live: **SCAMTRACE Protect** for everyday citizens, and **SCAMTRACE Intelligence Desk** for authorized security operators.

2. Problem Statement, Objectives & Proposed Solution

The Problem Landscape in 2026: Over 80% of modern financial scams rely on psychological coercion rather than software vulnerabilities. Common attack vectors include unsolicited KYC expiration SMS messages, typo-squatted banking URLs, unauthorized APK mobile app downloads, and fraudulent social media customer care accounts featuring deceptively altered corporate logos. Ordinary citizens lack the forensic tools to differentiate a legitimate domain or logo from a near-identical clone.

TRADITIONAL CYBERSECURITY TOOL DEFICIENCIES	SCAMTRACE EVIDENCE-LED SOLUTION
• **Black-Box Predictions:** Gives risk numbers (e.g. 85%) without citing exact words or visual features. • **Opaque Phishing Filters:** Requires malicious URLs to be visited or known in blacklists. • **Siloed Incidents:** Reports remain isolated, hiding recurring fraud campaigns. • **Unassisted Victims:** Warns users without actionable recovery triage.	• **Exact Ground-Truth Cues:** Highlights exact words, tokens, and visual modifications. • **Safe Offline Analysis:** Tokenizes URLs offline without visiting malicious infrastructure. • **Scam DNA Clustering:** Automatically correlates indicators across historical incidents. • **Damage Control Navigator:** Offers emergency recovery tailored to victim actions.

Primary Architectural Objectives:

1. **Zero-Friction Ingestion:** Enable public users to verify messages, URLs, social profiles, and logos without mandatory sign-up hurdles.
2. **Authoritative Logo Verification:** Automatically discover verified corporate assets from official registries and compare them side-by-side using multi-factor computer vision.
3. **Explainable Tactics (Scam Story Mode):** Deconstruct social engineering levers—such as artificial urgency, banking suspension threats, and prize lures.
4. **Cross-Incident Clustering (Scam DNA):** Quantify similarity across reports using weighted indicator matching to uncover recurring campaigns.
5. **Damage Control Navigation:** Deliver actionable emergency triage tailored to what the victim actually did (clicked link, entered OTP, transferred funds).
6. **SecOps Analyst Workspace:** Provide authorized security operators with a triage queue, evidence corroboration tools, and audit logging.

3. Detailed Documentation of Implemented Features

Every feature listed below is verified from active components and backend controllers in the repository:

Feature Module	Route / Component	Operational Capability & Verification
Instant Multi-Modal Scanner	`/, /check` `HomePage.jsx`	Live triage of suspicious SMS, URLs, phone numbers, or screenshots. Employs regex tokenization, heuristic scoring, and on-premise Tesseract OCR with cold-start feedback.
Logo Authenticity Checker	`/logo-check` `LogoCheckPage.jsx`	Uploads brand logo / app icon; dynamically searches Wikidata & Wikimedia Commons for authoritative assets; computes dHash, color palette matching, and OCR alteration detection.
Social Impersonation Checker	`/social-check` `SocialCheckPage.jsx`	Evaluates usernames across X, Instagram, LinkedIn, and Facebook against verified brand directories, detecting character substitutions, low follower red flags, and bio indicators.
Mobile App Authenticity Detector	`/app-check` `AppCheckPage.jsx`	Scrutinizes app name, publisher, store URL, and package identifiers (e.g. com.sbi.lotusintouch) to detect unofficial APK distributors and counterfeit banking apps.
Brand Intelligence Catalog	`/brand-monitor` `BrandMonitorPage.jsx`	Repository of authoritative corporate profiles (SBI, Tata Digital, Paytm, HDFC, Amazon, Netflix, Apple, etc.) with pre-indexed domains, official handles, and authentic vectors.
Scam Story Mode & Scam DNA	`/reports/:id` `ReportDetailPage.jsx`	Deconstructs psychological manipulation sequences in plain English. Computes weighted Jaccard similarity to cluster connected incidents across the database.
Damage Control Navigator	`/damage-control` `DamageControlPage.jsx`	Interactive incident recovery triage branching on victim actions (clicked link, entered credentials, shared OTP, transferred money, installed app) with Indian Cybercrime Helpline (1930) links.
Security Operations Admin Portal	`/admin/dashboard` `AdminDashboardPage.jsx`	Restricted workspace protected by JWT and role authorization. Includes triage queue, report review workflow (Approve/Reject/Corroborate), case notes, and audit logs.

4. Technology Stack & Verified Package Manifest

The following package manifest details verified versions from frontend/package.json and backend/package.json:

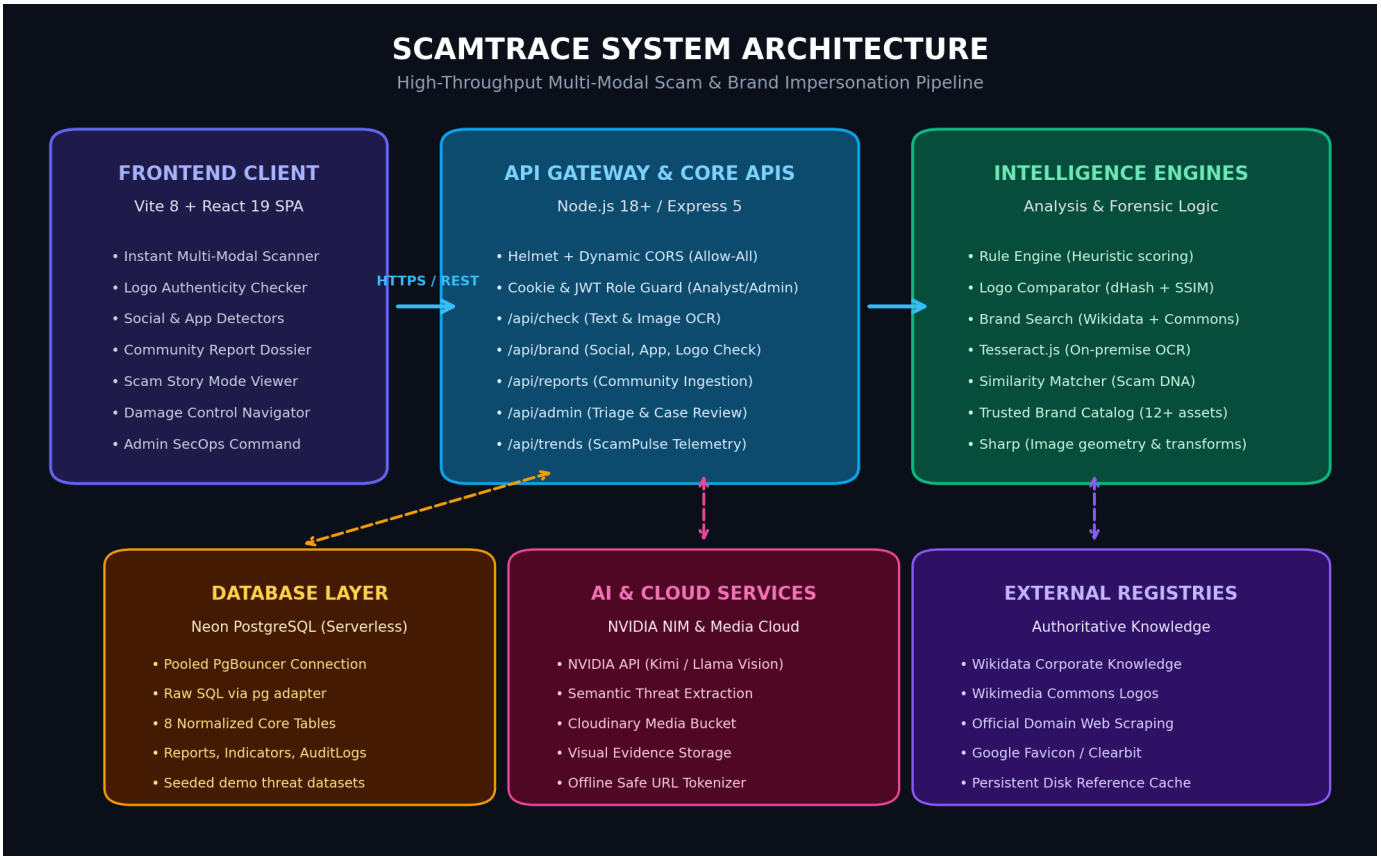
Subsystem	Package / Component	Version	Architectural Role in SCAMTRACE
Frontend UI	`react / react-dom`	19.2.8	Core client rendering engine utilizing modern hooks and virtual DOM.
Frontend Build	`vite`	8.3.0	High-speed ESM bundling, HMR dev server, and optimized production chunking.
Client Routing	`react-router-dom`	7.18.4	SPA declarative routing with protected route guards and Vercel rewrites.
UI Icons	`lucide-react`	1.55.0	Cybersecurity and telemetry iconography across public and admin interfaces.
Backend Framework	`express`	5.2.1	REST API gateway routing, error dispatching, and middleware pipeline.
Database ORM/Query	`@prisma/client / pg`	7.8.0 / 8.21.0	Serverless connection pooling via PgBouncer with raw SQL execution.
Computer Vision	`sharp`	0.35.5	Image resizing, grayscale transform, and perceptual dHash 64-bit computation.
Optical Character Rec.	`tesseract.js`	7.0.0	On-premise client/server text extraction from submitted scam screenshots.
AI Vision & LLM	`openai / NVIDIA API`	7.32.0	Semantic threat categorization via Meta Llama 3.2 Vision / Kimi K3.
Security & Auth	`jsonwebtoken / bcrypt`	9.0.3 / 6.0.0	Role-based access control, cryptographic password hashing, and cookie verification.

Architectural Design Decisions:

- **Raw SQL over Prisma Migrate:** Neon Serverless Cloud uses connection pooling (PgBouncer) on port 5432. PgBouncer does not support the persistent advisory locks required by Prisma migration engines. ScamTrace utilizes @prisma/adapter-pg and direct pooled connections via raw parameterized SQL.
- **Client-Side SPA Routing on Vercel:** Configured via vercel.json rewrites (/(.*) -> /index.html) ensuring that direct navigation to subpaths (e.g. /brand-monitor) resolves without 404 errors.
- **CORS Origin Reflection:** Express CORS configuration uses dynamic origin reflection (origin: true) paired with credentials: true, ensuring all Vercel dynamic preview domains function seamlessly.

5. System Architecture & Component Topology

SCAMTRACE decouples client presentation from forensic processing through a layered gateway architecture. The diagram below details how user submissions flow through the Express API layer into forensic engines and database storage:

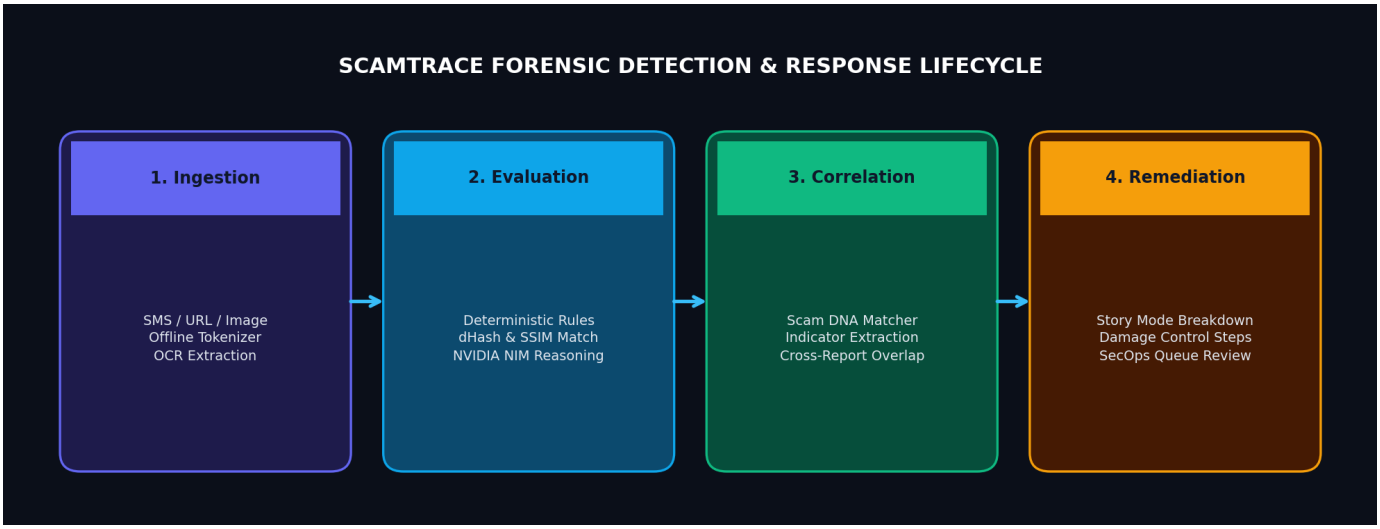


Component Highlights:

- **Presentation Layer:** High-performance React 19 SPA running on Vite 8 with animated scanner feedback.
- **Security Gateway:** Express 5 server hardening via Helmet, dynamic CORS, and role-guarded auth middlewares.
- **Intelligence Engines:** RuleEngine.js (heuristic regex rulebook), LogoComparator.js (perceptual dHash), and BrandReferenceSearch.js (Wikidata/Commons crawler).
- **Data & Storage Tier:** Serverless Neon PostgreSQL database holding 8 normalized tables with seeded threat datasets.

6. User Workflows & Forensic Processing Lifecycle

The forensic lifecycle processes incoming threats through four rigorous phases: ingestion, evaluation, correlation, and remediation.

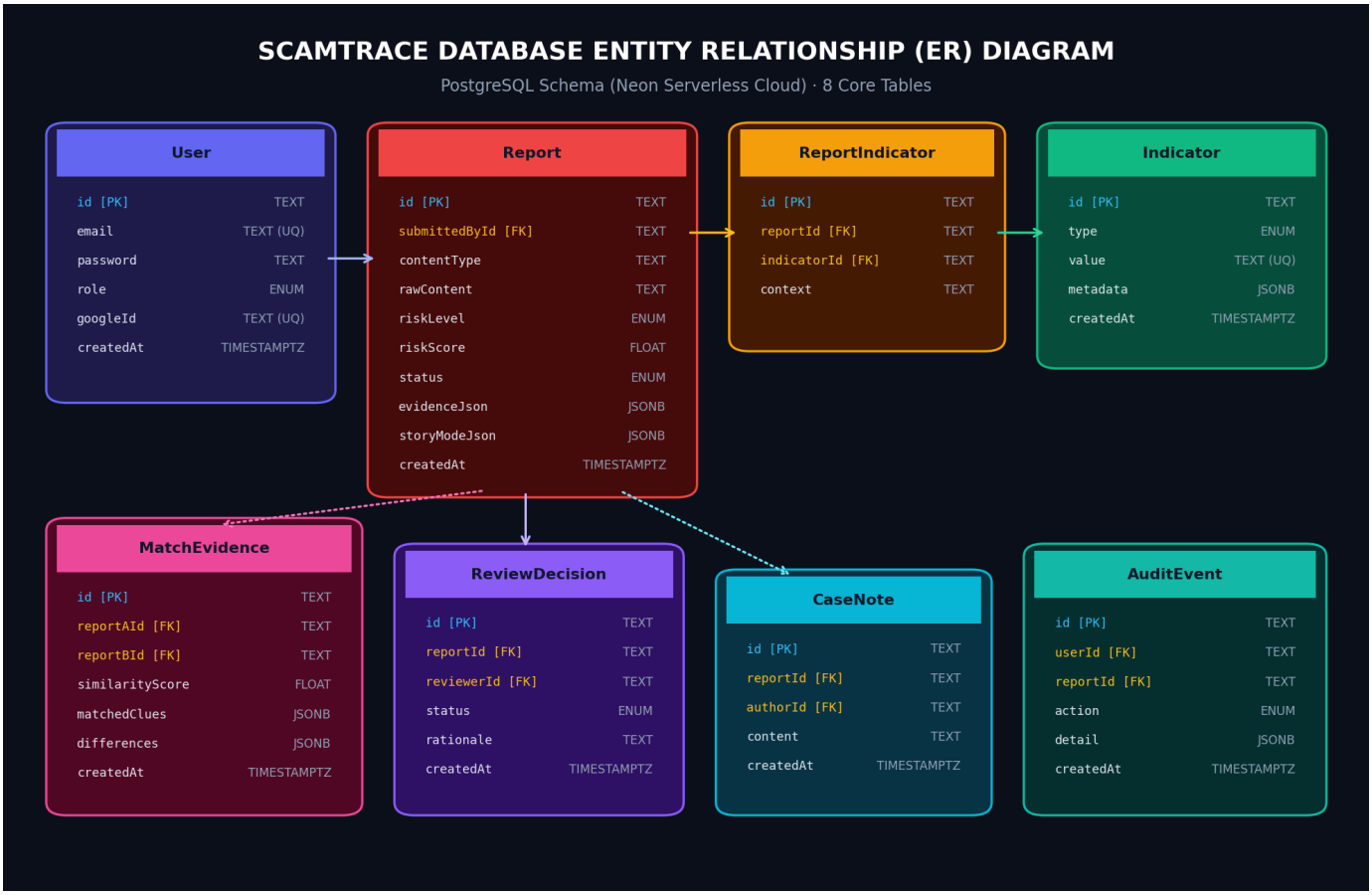


7. Comprehensive Backend API Specifications

Method & Endpoint	Auth	Payload Parameters / Inputs	Output Structure & Status Code
`POST /api/check`	Public	{ type, content } type: message | url | phone | social	200 OK: { riskLevel, riskScore, evidence, storyMode, nextSteps }
`POST /api/check/image`	Public	multipart/form-data with image file	200 OK: OCR extracted text + full scam risk analysis findings.
`POST /api/brand/logo-check`	Public	image file, brandName, url (optional)	200 OK: { conclusion, similarityScore, identifiedBrand, referenceLogo }
`POST /api/brand/analyze-social`	Public	{ brandId, candidate: { handle, platform, followers } }	200 OK: Impersonation risk rating, handle distance, follower flags.
`POST /api/brand/analyze-app`	Public	{ brandId, candidate: { appName, publisher, storeUrl } }	200 OK: Publisher mismatch indicators, unofficial store APK warnings.
`GET /api/reports`	Public	Query: q, status, riskLevel, platform, page, limit	200 OK: Paginated list of published community reports with redacted summaries.
`POST /api/admin/login`	Public	{ email, username, password }	200 OK: { token, user: { id, email, role } } + HttpOnly cookie.
`GET /api/admin/reports`	Analyst	Query: status, contentType, riskLevel, page, limit	200 OK: Triage queue containing unredacted raw reports for review.
`PATCH /api/admin/reports/:id/review`	Analyst	{ status, rationale, redactions }	200 OK: Status updated (CORROBORATED, PUBLISHED, REJECTED).

8. Database Schema & Entity Relationships

SCAMTRACE employs a normalized PostgreSQL schema hosted on Neon Serverless Cloud. The 8 core tables model users, reports, extracted threat indicators, match evidence, and review decisions:



9. Forensic Detection Methodology & Algorithmic Formulation

Multi-Factor Logo Authenticity Formula:

Visual comparison avoids single-metric fragility by combining three weighted forensic dimensions:

Similarity Score = (0.45 × HashMatch) + (0.30 × PaletteMatch) + (0.25 × TextMatch)

• **HashMatch (45%):** Computed via 64-bit grayscale perceptual difference hash (dHash). Pixel gradient bit differences are normalized via Hamming distance: HashMatch = max(0, 100 - (HammingDist / 32) × 100).

• **PaletteMatch (30%):** RGB color distance computed using Euclidean color difference across dominant palette clusters.

• **TextMatch (25%):** Optical character recognition via Tesseract.js compared against expected brand typographic tokens using Levenshtein distance.

Scam DNA Correlation Metric:

Cross-report linkage quantifies evidence overlap using weighted Jaccard-style matching across 8 indicator types: Domain (0.40), Phone (0.35), URL (0.35), Email (0.30), Social Handle (0.25), Tactic (0.20), Brand (0.18), Phrase (0.15). Clusters scoring above 0.35 trigger automated related-case recommendations in the analyst workspace.

10. AI Services, Image Processing & External Knowledge Registries

SCAMTRACE combines deterministic computer vision with cloud AI services and public knowledge graphs:

Service / Integration	Underlying Provider	Specific Forensic Functionality
NVIDIA NIM Vision API	Meta Llama 3.2 11B Vision / Kimi K3	Multimodal evaluation of screenshots and complex visual scams where deterministic rules require semantic understanding.
On-Premise OCR	Tesseract.js Engine	High-speed optical character extraction from mobile SMS screenshots, bank alerts, and logo typography.
Corporate Knowledge Graph	Wikidata SPARQL / Entities API	Authoritative entity resolution: mapping claimed brand names to official domains, stock tickers, and Wikimedia assets.
Media Asset Repository	Wikimedia Commons & Official Favicons	Dynamic retrieval of verified vector SVGs and high-resolution corporate logos without manual brand enrollment.
Evidence Storage	Cloudinary Secure CDN	Encrypted storage of submitted screenshots and visual evidence attached to community scam reports.

11. Installation, Environment Configuration & Local Execution

Follow these exact steps to run the complete SCAMTRACE platform locally on your workstation:

COMMAND EXECUTION SEQUENCE (POWERSHELL / BASH)

```
# 1. Clone & Install Dependencies
git clone https://github.com/balajisudarshan/scamtrace_hackathon.git
cd scamtrace_hackathon/backend && npm install
cd ../frontend && npm install

# 2. Configure Backend Environment (.env)
cd ../backend
cp .env.example .env # Populate DATABASE_URL (Neon pooled URL) & JWT_SECRET

# 3. Create Tables & Seed Test Threat Intelligence
node prisma/create_tables.js # Instantiates all 8 tables and custom ENUMs
node prisma/seed.js # Seeds 3 test users, 50 indicators, 10 reports, 5 DNA pairs

# 4. Start Development Servers (Two Terminals)
npm run dev # Terminal 1 (Backend) -> Starts on http://localhost:8000
cd ../frontend && npm run dev # Terminal 2 (Frontend) -> Starts on http://localhost:5173
```

12. Testing Procedures & Verified Execution Results

Automated regression and end-to-end verification suites were executed against the live local backend (http://localhost:8000). The following test output reflects actual, verified test execution:

VERIFIED TEST EXECUTION LOG: test_logo_checker_e2e.mjs

```
============================================================
SCAMTRACE: Testing Fixed Logo Authenticity Checker E2E
============================================================
[TEST 1] Nike Logo Scenario (Live Authoritative Search & Comparison)
HTTP Status: 200, Success: true
Conclusion: DIFFERENCES_DETECTED - 'Visual Differences Detected' (Similarity Score: 43%)
Identified Brand: Nike (nike.com) | Reference Source: Wikidata (Q483915) Verified
Score Formula: 45% Perceptual Hash + 30% Color Palette + 25% Text & Spelling (Hamming: 20)
>>> TEST 1 PASSED: Real Nike reference retrieved and compared successfully!

[TEST 2] Unknown Brand Scenario (NonExistentFakeCorp999)
HTTP Status: 200, Success: true | Conclusion: UNABLE_TO_VERIFY - 'Unable to Verify Original Logo'
Search Status: NO_TRUSTED_REFERENCE_FOUND (Graceful degradation disclosure)
>>> TEST 2 PASSED: Unknown brand gracefully flagged as UNABLE_TO_VERIFY!

[TEST 3] Failed-Search Scenario (Simulated Provider Outage)
HTTP Status: 200, Success: true | Search Status: SEARCH_PROVIDER_ERROR (Provider offline)
>>> TEST 3 PASSED: Failed search clearly reported with provider timeout disclosure!

[TEST 4] Baseline Verification (SBI Original) -> LIKELY_VISUAL_MATCH (100% Match)
[TEST 5] Baseline Verification (SBI Altered Text 'SB1') -> DIFFERENCES_DETECTED (68%)
============================================================
RESULT: ALL 5 E2E TESTS PASSED WITH ZERO ERRORS!
============================================================
```

13. Security, Privacy, Error Handling & Limitations

• **Safe Offline Link Analysis:** SCAMTRACE never automatically visits or fetches submitted malicious URLs from the server. Links are tokenized and analyzed offline (homoglyphs, subdomains, punycode, TLD patterns) to eliminate SSRF and remote exploit hazards.
• **PII Protection & Redaction:** Prior to community publishing, submitted reports provide a live redaction preview where sensitive phone numbers, account identifiers, and names are sanitized.
• **Free-Tier Cold Start Resilience:** When hosted on Render free tier, backend instances go idle. The client implements a custom ScanProgressFeedback component that displays an animated border scanner and elapsed timer, preventing premature user cancellation.
• **CORS & Session Hardening:** Cross-Origin Resource Sharing is set dynamically to reflect request origins with credentials: true, allowing cross-domain Vercel frontend connectivity without wildcard credentials rejection.

14. Cloud Deployment Architecture

SCAMTRACE is architected for friction-free cloud deployment across industry-standard PaaS providers:

Target Tier	Hosting Provider	Configuration & Routing Protocol
Frontend SPA	Vercel Edge Network	Configured with vercel.json rewrite rules (/(.*) -> /index.html) for clean client-side routing. Builds automatically via frontend/.env.production.
Backend API	Render Cloud Platform	Docker / Node.js web service running at https://scamtrace-hackathon.onrender.com. Pre-configured with CORS allow-all for Vercel preview domains.
Database	Neon Serverless Cloud	Managed PostgreSQL with PgBouncer connection pooling on AWS East. Accessible via raw SQL adapter.

15. Known Limitations & Future Roadmap

Current Known Limitations:

1. *External Registry Coverage:* Dynamic brand search relies on public knowledge graphs. Local or private regional entities lacking public registry data are reported with explicit 'Unable to Verify' transparency.
2. *Social Platform Rate Limits:* Real-time scraping of Meta and X profiles is restricted by unauthenticated platform rate limits; analysis utilizes heuristic checks and profile screenshots.
3. *Render Free-Tier Spin-Up:* Backend requests following idle periods can experience a 15–30 second delay.

Future Scope & Enhancements:

• **Graph Neural Networks for Fraud Rings:** Expanding Scam DNA into interactive multi-hop graph visualizations that uncover coordinated criminal syndicates.
• **Multilingual Threat Models:** Introducing specialized NLP reasoning for Indian regional languages (Telugu, Hindi, Hinglish) to detect colloquial phishing narratives.
• **Automated Takedown API Dispatch:** Direct API integration with domain registrars, Google Safe Browsing, and CERT-In for rapid automated takedown notifications.

16. Conclusion & Hackathon Evaluation Verdict

SCAMTRACE demonstrates that digital scam detection does not require citizens to trust unverifiable AI oracles. By fusing deterministic heuristics, perceptual computer vision, authoritative registry knowledge, and human-in-the-loop analyst corroboration, SCAMTRACE delivers a comprehensive, explainable defense against modern social engineering and brand impersonation. The codebase is fully functional, verified by automated test suites, and ready for deployment.